AI-CAD House Builder: A Tool-Using Large Language Model Framework for Natural-Language Architectural CAD

SHYAMSUNDAR KAMALAKANNAN– 24623126

DHASARADHAN D – 24623143

PRAJESHWARAN B – 24623139

BATCH NO. 16

PROJECT REPORT

Submitted to the

SCHOOL OF ENGINEERING

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

in Partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE)

VELS INSTITUTE OF SCIENCE, TECHNOLOGY AND ADVANCED STUDIES (VISTAS)

CHENNAI – 600 117

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project report "**AI-CAD House Builder: A Tool-Using Large Language Model Framework for Natural-Language Architectural CAD**" is the bonafide work of **SHYAMSUNDAR KAMALAKANNAN-24323126**,**DHASARADHAN D-24623143**, **PRAJESHWARAN B – 24623139** who carried out the project under my supervision. Certified further that, to the best of my knowledge, the work reported here in does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE OF THE HOD
Dr. K. ULAGA PRIYA, M.E., Ph.D.
Associate Professor & Head
Department of AI&ML
School of Engineering
VISTAS, Chennai

SIGNATURE OF THE SUPERVISOR
DR. G.Revathy
Assistant Professor
Department of AI&ML
School of Engineering
VISTAS, Chennai

Submitted for the Project Work and Viva-Voce examination held on _____ at Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai – 600 117.

INTERNAL EXAMINER **EXTERNAL EXAMINER**

DECLARATION

We hereby declare that the project report entitled " **AI-CAD House Builder: A Tool-Using Large Language Model Framework for Natural-Language Architectural CAD**, submitted to Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Information Technology, is a record of original work carried out by us under the guidance of **Dr. G.Revathy**, Professor, Department of AI&ML. The work has not been submitted, in part or in full, for the award of any other degree or diploma.

Place: Chennai

Date:

SHYAMSUNDAR KAMALAKANNAN– 24623126

DHASARADHAN D – 24623143

PRAJESHWARAN B – 24623139

ACKNOWLEDGEMENT

It is a great pleasure and privilege to acknowledge the help and support we received in completing this project. The infrastructure and laboratory facilities provided by our **FOUNDER & CHAIRMAN, Shri. Dr. ISHARI K. GANESH**, have been a constant source of motivation, and we take this opportunity to thank him.

We are grateful to our **HEAD OF THE DEPARTMENT, Dr. K. ULAGA PRIYA**, for her continuous support and encouragement throughout the project. We extend our sincere thanks to our **SUPERVISOR, Dr. G.Revathy, Professor, Department of AI&ML**, who patiently guided us and gave us valuable advice at every stage of the work.

We also thank the faculty members, librarian and non-teaching staff of the department for their help. Finally, we thank our parents and friends for their support and encouragement, without which this project would not have been possible.

SHYAMSUNDAR KAMALAKANNAN– 24623126
DHASARADHAN D – 24623143
PRAJESHWARAN B – 24623139

ABSTRACT

AI-CAD House Builder is a natural-language architectural CAD assistant that turns a plain-language description of a house into an editable single-floor model. Conventional CAD tools are precise but require training in commands, layers and dimensions, while large language models understand such descriptions well but are unreliable when asked to produce geometry directly. The project separates these roles: a language model running on the user's own laptop plans the house as structured data, and deterministic code builds and displays the geometry.

The frontend is built with React and Vite, with a 2D blueprint editor and a 3D scene built with Three.js and React Three Fiber. A FastAPI backend exposes generate, modify, analyze and health endpoints. Qwen 2.5 7B Instruct, served locally by Ollama, returns the house as JSON describing walls, rooms, doors, windows, floors and services; the backend normalises this output to a single floor, and a keyword-based generator takes over when the model fails or times out. The prototype also provides click-and-drag editing of walls and openings, a first-person walkthrough, electrical and plumbing planning, a cost estimate in rupees, undo and redo, and JSON, CSV, DXF and PDF output, all without a cloud service.

Generation, AI-assisted modification, 2D and 3D editing and export work in the current prototype. The typed tool registry and blocking validator specified in the design are not yet implemented: the model currently returns the whole house in one response, and design checks are advisory. An evaluation that compares direct generation, planning without validation and planning with validation over seven test scenarios and seven metrics has been defined, and measured results will be reported once the experiments are run.

The project shows that a small local model, limited to producing structured data that code turns into geometry, can make house modelling accessible to non-experts while keeping designs private and fully editable.

Keywords: computer-aided design, large language models, natural-language interfaces, floor-plan generation, Qwen 2.5, Ollama, local inference, FastAPI, Three.js, React.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	i
	LIST OF FIGURES AND TABLES	iii
	LIST OF ABBREVIATIONS	iv
1	**CHAPTER 1 – INTRODUCTION**	1
	1.1 Overview of the project	1
	1.2 Scope and objective	2
2	**CHAPTER 2 – REVIEW OF THE LITERATURE**	3
	2.1 Literature overview	3
	2.2 Research summary	3
3	**CHAPTER 3 – SYSTEM DESIGN**	5
	3.1 Existing system	5
	3.2 Proposed system	5
	3.3 System architecture	6
	3.4 Data flow / workflow	7
	3.5 Hardware requirements	8
	3.6 Software requirements	8
4	**CHAPTER 4 – IMPLEMENTATION**	10
	4.1 Methodology	10
	4.2 Implementation	10
	4.2.1 House Data Model	10
	4.2.2 Backend API Module	11
	4.2.3 Local Language-Model Planner	11
	4.2.4 Normalisation and Fallback Module	11
	4.2.5 2D Plan Editor Module	12
	4.2.6 3D Scene and Walkthrough Module	12
	4.2.7 Services, Cost and Design-Check Module	12
	4.2.8 History, Persistence and Export Module	12
	4.3 Module integration	12
5	**CHAPTER 5 – RESULTS, CONCLUSION AND FUTURE WORK**	14
	5.1 Results and evaluation	14
	5.2 Discussion	17
	5.3 Challenges and limitations	17
	5.4 Conclusion	18

CHAPTER NO.	TITLE	PAGE NO.
	5.5 Future work	18
	REFERENCES	19

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
3.1	Use-case diagram of AI-CAD House Builder	6
3.2	System architecture of AI-CAD House Builder	7
3.3	Generate workflow with fallback (as implemented)	8
4.1	Structure of the house model	10
4.2	Frontend–backend interaction for an AI request	13
5.1	House model returned by /generate in the set-up test	14
5.2	Main workspace with the starter 2BHK in the 2D blueprint view	15
5.3	The same house in the 3D view	15
5.4	Inspector and drag handles on a selected wall	15
5.5	First-person walkthrough	16

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
2.1	Summary of related studies	4
3.1	Existing approaches to house modelling	5
3.2	Features of the proposed system and implementation status	5
3.3	Hardware requirements	8
3.4	Software requirements	8
4.1	Backend API endpoints	11
5.1	Observed results from the running application	14
5.2	Evaluation metrics	16
5.3	Test scenarios	16

LIST OF ABBREVIATIONS

ABBREVIATION	FULL FORM
2BHK	Two Bedrooms, Hall and Kitchen
AI	Artificial Intelligence
API	Application Programming Interface
BIM	Building Information Modelling
CAD	Computer-Aided Design
CORS	Cross-Origin Resource Sharing
CPU	Central Processing Unit
CSV	Comma-Separated Values
DXF	Drawing Exchange Format
GAN	Generative Adversarial Network
GPU	Graphics Processing Unit
HTTP	Hypertext Transfer Protocol
IFC	Industry Foundation Classes
INR	Indian Rupee
JSON	JavaScript Object Notation
LLM	Large Language Model
MEP	Mechanical, Electrical and Plumbing
PAL	Program-aided Language Model
R3F	React Three Fiber
RAM	Random Access Memory
REST	Representational State Transfer
UI	User Interface
WebGL	Web Graphics Library

CHAPTER 1 – INTRODUCTION

1.1 OVERVIEW OF THE PROJECT

Introduction

Computer-Aided Design (CAD) is the standard way to draw buildings, but it asks a lot of the person using it. Even a simple two-bedroom house has to be drafted wall by wall, with commands, layers and coordinates, and every change of mind means repeating the edits. Large Language Models (LLMs) understand a request such as "a 10 m × 8 m house with two bedrooms and an attached bath" very well, but when they are asked to produce the geometry directly they can return rooms that overlap, walls that do not meet and doors that sit outside any wall [1], [2].

AI-CAD House Builder brings these two strengths together. A language model running locally on the student's laptop (Qwen 2.5 7B Instruct, served by Ollama) reads the request and plans the house as structured data: walls, rooms, doors, windows, floors and services [3], [4]. Deterministic code then builds the geometry from that data and shows it as an editable 2D plan and an interactive 3D scene. The guiding rule of the project is that **the language model plans, and code draws**.

Problem Statement

A person who wants to turn a house idea into an editable model currently has to choose between precision and ease of use:

- **CAD and BIM software** is precise but has a steep learning curve, and every revision repeats manual drafting.
- **Learning-based floor-plan generators** produce plausible layouts, but from bubble diagrams or boundaries rather than from conversation, and a change usually means generating a new plan [5], [6].
- **LLM output used directly as geometry** can be invalid: overlapping rooms, open walls and openings that do not lie on a wall [1], [2].
- **Cloud AI services** need a reliable internet connection and send every design to an external server.

The problem addressed by this project is to build an assistant that turns **plain-language house requirements into a valid, editable 2D and 3D model**, while **running entirely on a local machine**.

System Overview

AI-CAD House Builder integrates five components:

- **Web interface (React and Vite):** a prompt bar, a 2D blueprint editor, an inspector panel and planner modes for electrical and plumbing layouts [7], [8].
- **3D scene (Three.js, React Three Fiber and Drei):** an extruded model of the same house, with a first-person walkthrough.

- **Backend (FastAPI):** the `/generate`, `/modify`, `/analyze` and `/health` endpoints, output normalisation and a keyword-based fallback generator.
- **Local language model (Ollama, qwen2.5:7b-instruct):** turns a request, or a request plus the current house, into house JSON.
- **House model (JSON):** one structure that drives editing, rendering, cost estimation and export.

Advantages of the System

- **Lower learning curve:** a first design comes from one sentence instead of manual drafting.
- **Geometry built by code:** the model only supplies structured data; walls, openings and the 3D scene are constructed by deterministic code.
- **One consistent model:** the 2D plan, 3D scene, walkthrough, cost estimate and exports all read the same house JSON.
- **Local and private:** prompts and designs never leave the laptop, and there is no API cost.
- **Open-source stack:** Python, FastAPI, React, Vite, Three.js and Ollama carry no licence cost.

1.2 SCOPE AND OBJECTIVE

Purpose of the Study

The purpose of the project is to design and implement a locally run assistant that keeps the language model in the role of planner and leaves geometry to deterministic code, so that a non-expert can describe a house, see it in 2D and 3D, and keep editing it.

Objectives

1. To generate an initial single-floor house design from a natural-language prompt.
2. To represent the house as a structured JSON model that deterministic code renders in 2D and 3D.
3. To restrict the language model to planning, so that it never draws geometry itself.
4. To validate operations before they change the model, through a typed tool registry and validator (Phase II).
5. To support both manual editing and AI-assisted modification of the same model.
6. To run the complete system on one laptop with a quantised 7B model and no cloud service.
7. To evaluate the approach against baselines with measured metrics.

Scope of the Project

- **Functional scope:** single-floor residential houses; rooms, walls, doors, windows and roof; 2D editing; 3D view and walkthrough; electrical and plumbing planning; cost estimate in INR; undo and redo; save and load; JSON, CSV, DXF and PDF output.

- **Technical scope:** React and Vite on the frontend; Three.js with React Three Fiber and Drei for 3D; Python and FastAPI on the backend; Qwen 2.5 7B Instruct served by Ollama.
- **Out of scope:** multi-floor houses, structural analysis, building-code compliance checking, curved walls, furniture libraries, BIM/IFC exchange and user accounts. These are future work (Section 5.5).

CHAPTER 2 – REVIEW OF THE LITERATURE

2.1 LITERATURE OVERVIEW

This chapter reviews the work that shapes AI-CAD House Builder. It covers learning-based floor-plan generation, language-guided design, generation of parametric CAD from text, and tool-augmented language models that hand exact work to software tools. It ends by identifying the research gap that the project addresses.

2.2 RESEARCH SUMMARY

Learning-Based Floor-Plan Generation

Wu et al. built RPLAN, a dataset of more than 80,000 real residential plans, and generated interior layouts in stages: room locations first, then walls [9]. Nauata et al. proposed House-GAN, a relational GAN that turns a bubble diagram of room adjacencies into a layout [10], and House-GAN++, which refines a layout iteratively by feeding the previous result back as a constraint [5]. Hu et al. presented Graph2Plan, which fits a layout graph into a given building boundary [11]. Sun et al. proposed WallPlan, which generates a floor plan as a graph of walls [12], and Shabani et al. proposed HouseDiffusion, a diffusion model that outputs vector room and door polygons [6]. These systems produce convincing plans, but their input is a graph or a boundary rather than a conversation, and a change usually means generating the plan again.

Language-Guided Design

Leng et al. introduced Tell2Design, a dataset of more than 80,000 floor plans paired with natural-language instructions, together with a sequence-to-sequence baseline [13]. Their benchmark showed that text-to-image generators struggle with the strict spatial constraints that a usable plan must satisfy, which supports the idea of producing structured output rather than pictures.

Text to Parametric CAD

Wu et al. introduced DeepCAD, a network that generates CAD models as sequences of sketch and extrude commands [14]. Khan et al. extended this to text with Text2CAD, which generates parametric models from prompts ranging from beginner to expert level [15]. Li et al. presented CAD-Llama, which adapts a pretrained LLM to write parametric CAD models as Python-like structured code [16]. These methods target individual mechanical parts; they have no notion of rooms, walls or openings.

Tool-Augmented and Verified Generation

Schick et al. showed with Toolformer that a language model can learn when to call external tools [17]; Yao et al. interleaved reasoning with actions in ReAct [18]; and Gao et al. showed with PAL that a model which writes a program for an interpreter to run outperforms one that computes in text [19]. In CAD, Mallis et al. built CAD-Assistant, in which a vision-language planner issues commands to FreeCAD through its Python API and checks their effect on the design [20]. Alrashedy et al. proposed CADCodeVerify, in which a vision-language model

asks and answers validation questions about the object its code produced and then corrects the code [2]. The common lesson is that the model should plan while deterministic tools execute and check.

Local Language Models

Qwen2.5 is an open-weight model family from 0.5B to 72B parameters [3]. Quantised, the 7B instruct model runs on a laptop through Ollama or llama.cpp [4], [21], which makes a fully local assistant practical.

Table 2.1: Summary of related studies

Ref.	Study	Key idea	Relevance to AI-CAD House Builder
[9]	Wu et al., 2019 (RPLAN)	80k real residential plans; staged generation	Shows what realistic layouts look like
[5]	Nauata et al., 2021 (House-GAN++)	Iterative layout refinement from bubble diagrams	Layouts without conversation
[12]	Sun et al., 2022 (WallPlan)	Plan generated as a wall graph	Walls as the primary representation
[6]	Shabani et al., 2023 (HouseDiffusion)	Diffusion over vector polygons	Vector output, but no editing dialogue
[13]	Leng et al., 2023 (Tell2Design)	Floor plans from language instructions	Language input; structured output needed
[15]	Khan et al., 2024 (Text2CAD)	Text to parametric CAD sequences	Text-driven CAD, mechanical parts only
[16]	Li et al., 2025 (CAD-Llama)	LLM writes structured CAD code	Structured output from an LLM
[20]	Mallis et al., 2025 (CAD-Assistant)	LLM planner calls FreeCAD tools	Planner + tools, but generic CAD
[2]	Alrashedy et al., 2025 (CADCodeVerify)	Validation questions refine CAD code	Checking output before accepting it
[19]	Gao et al., 2023 (PAL)	Interpreter does the exact work	Core motivation: model plans, code executes

Research Gap

- Floor-plan generators are not conversational and do not support targeted edits such as "make Bedroom 2 wider".
- Text-to-CAD methods produce single mechanical parts, not houses made of rooms, walls and openings.
- Tool-augmented CAD agents rely on large cloud models; CAD-Assistant is built on GPT-4o [20] and CADCodeVerify was evaluated mainly with GPT-4 [2].
- No single tool combines language input, a persistent editable house model, a 2D and 3D workspace and fully local operation.

AI-CAD House Builder addresses these gaps by combining a local language-model planner, a structured house model built into geometry by code, and an editable 2D and 3D workspace in one application.

CHAPTER 3 – SYSTEM DESIGN

3.1 EXISTING SYSTEM

People who want to model a house today choose between the categories in Table 3.1.

Table 3.1: Existing approaches to house modelling

Category	Strength	Limitation for a non-expert
Commercial CAD/BIM (AutoCAD, Revit, SketchUp)	Precise drafting; industry formats	Steep learning curve; every revision is manual
Learning-based floor-plan generators (House-GAN++, HouseDiffusion)	Plausible layouts in seconds	Graph or boundary input; no conversation; edits regenerate the plan
Text-to-CAD models (Text2CAD, CAD-Llama)	Parametric output from text	Mechanical parts; no rooms, walls or openings
Cloud LLM chat assistants	Understand plain-language requests	No geometric guarantees; internet required; designs leave the device

Limitations of the Existing System

- **Expertise barrier:** CAD tools require training before an idea becomes geometry.
- **Slow revision:** changing a room size means repeating manual edits.
- **Invalid direct output:** language-model geometry can contain overlaps, open walls and misplaced openings [1], [2].
- **Fragmented workflow:** layout, 3D view, cost and export usually live in separate tools.
- **Privacy and cost:** cloud services need connectivity, charge per request and receive the user's designs.

3.2 PROPOSED SYSTEM

AI-CAD House Builder proposes a **locally run, planner-and-builder architecture**. The language model converts a request into a house model in JSON; the backend normalises that model; and the frontend builds the 2D plan and the 3D scene from it with deterministic code. The design also specifies a typed tool registry and a blocking validator between the planner and the model. Table 3.2 lists the features and their status in the current prototype, as established from the project source code.

Table 3.2: Features of the proposed system and implementation status

Feature	Status in current implementation
React/Vite workspace: prompt bar, 2D blueprint, 3D view	Implemented
House generation through POST /generate with Qwen 2.5 7B via Ollama	Implemented; the whole house is returned as one JSON object
Keyword-based fallback generator (builtin_house)	Implemented; used on timeout or invalid output, marked in the _source field
AI modification through POST /modify	Implemented; returns the whole house again, not a change set
Normaliser (_single_floor_house)	Implemented; coerces values, forces one floor, drops openings whose wallId does not exist

Feature	Status in current implementation
Design checks (/analyze, validateHouse)	Implemented as advisory checks: issues are listed, not blocked or corrected
Typed tool registry and blocking validator with retry	Not yet implemented (Phase II)
2D editing: click-to-place, drag handles, wall-length controls	Implemented
3D scene with roof types, materials and first-person walkthrough	Implemented
Electrical and plumbing planner modes; cost estimate in INR	Implemented
Undo/redo (50 steps); save and load	Implemented; browser storage and JSON files
Export: JSON, CSV, DXF, PDF/print	Implemented; DXF contains line entities only
Multi-floor houses	Not supported; the normaliser forces a single floor
llama.cpp as the model runtime	Planned; Ollama is used today

Figure 3.1 shows how a user works with the system. The user describes a house and clicks Generate, then refines the result either by hand in the 2D editor or by typing an instruction for the model. The same house can be viewed in 3D, walked through, given electrical and plumbing layouts, costed and exported. Only generation and AI modification involve the language model.

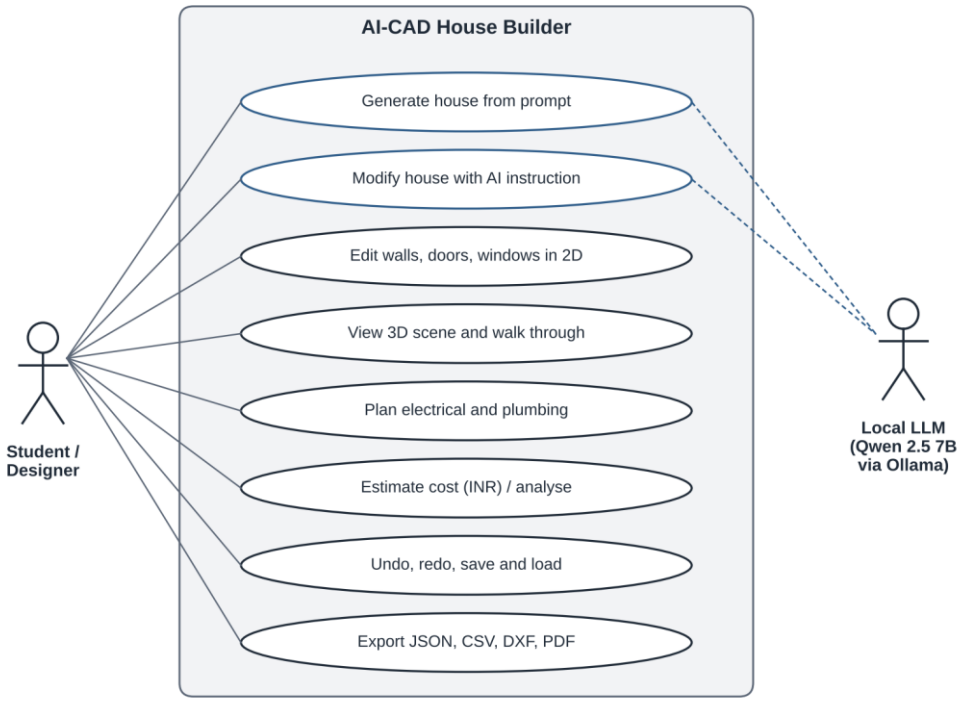


Figure 3.1: Use-case diagram of AI-CAD House Builder

3.3 SYSTEM ARCHITECTURE

AI-CAD House Builder runs as three local services, shown in Figure 3.2. The **frontend** is a React application served by Vite at localhost:5173. The **backend** is a FastAPI application at 127.0.0.1:8000 that exposes the REST endpoints, calls the model, normalises its output and falls back to a keyword-based generator when needed. The **language-model service** is Ollama

at 127.0.0.1:11434 running qwen2.5:7b-instruct. The house JSON is the single source of truth: every view and every export reads it. Components drawn with dashed outlines are part of the design but not yet built.

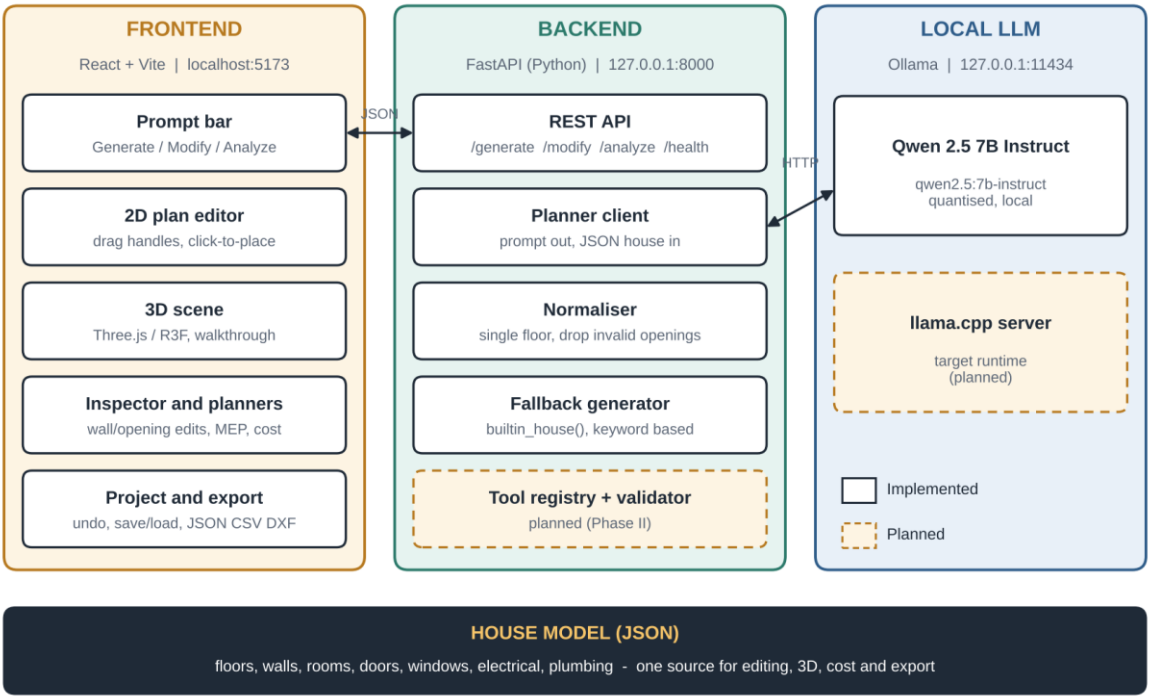


Figure 3.2: System architecture of AI-CAD House Builder

3.4 DATA FLOW / WORKFLOW

Figure 3.3 shows how one Generate request is processed. The backend sends the prompt to the model and checks whether a usable house JSON came back in time. If it did, the result is normalised to a single floor and returned with `_source` set to the model; otherwise the built-in keyword generator produces a layout and the response is marked as a fallback. The frontend stores the previous state in its undo history and redraws both views.

A Modify request follows the same path, except that the current house JSON is sent with the instruction and the model returns the complete updated house. An Analyze request does not involve the model: the backend checks room sizes and returns a list of issues, which the user may act on.

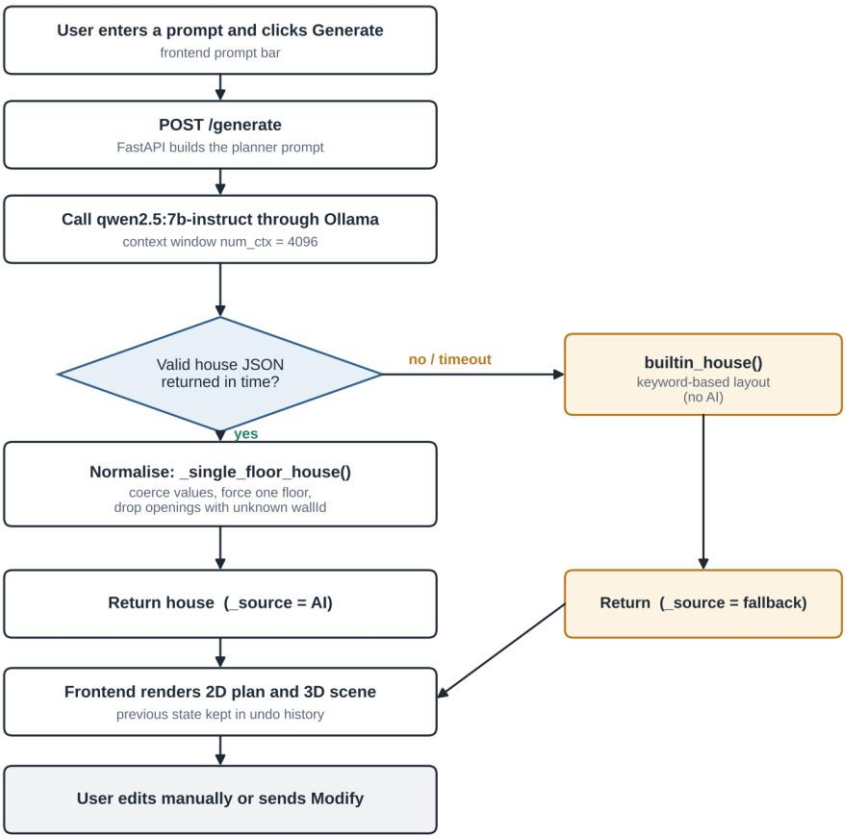


Figure 3.3: Generate workflow with fallback (as implemented)

3.5 HARDWARE REQUIREMENTS

Table 3.3: Hardware requirements

Component	Minimum	Recommended
Processor	64-bit quad-core CPU	Recent multi-core CPU
RAM	8 GB (quantised 7B model)	16 GB or more
Storage	About 10 GB free (model, Python environment, Node packages)	SSD
GPU	Not required; CPU inference is slower	GPU supported by Ollama
Display	Browser with WebGL support	Dedicated graphics for smooth 3D
Network	Only for the initial downloads	Broadband

The language model is the main computational load; the web application is lightweight. These figures are indicative and were not benchmarked.

3.6 SOFTWARE REQUIREMENTS

Table 3.4: Software requirements

Software	Purpose
Windows 10/11	Development machine; start_backend.bat and start_frontend.bat
Python 3 with a virtual environment (.venv)	Backend language

Software	Purpose
FastAPI and Uvicorn	REST API and server (uvicorn main:app --port 8000)
Ollama with qwen2.5:7b-instruct	Local model runtime and planner model
Node.js and npm	Frontend tooling (npm install --legacy-peer-deps)
React and Vite	User interface and development server
Three.js, React Three Fiber and Drei	3D scene and walkthrough
Web browser (Chrome or Edge)	Runs the application with WebGL
VS Code, Git	Development

Exact package versions are pinned in `backend/requirements.txt` and `frontend/package-lock.json`.

Software Modules

- **backend/main.py** (about 450 lines): FastAPI application, CORS, the four endpoints, the Ollama client, the normaliser and the fallback generator.
- **frontend/src/App.jsx** (about 2,600 lines): application state and undo history, 2D editor, inspector, 3D scene, walkthrough, planners, cost estimate and exports.
- **frontend/src/main.jsx, index.css:** entry point and styling.
- **start_backend.bat, start_frontend.bat, README.md:** one-click start-up and setup instructions.

CHAPTER 4 – IMPLEMENTATION

4.1 METHODOLOGY

The project followed a modular, incremental approach. Each part was built and checked on its own and then connected through the house JSON and the REST API. The stages were:

- **Requirement analysis:** identifying the need for language input, editable output and local operation.
- **Design:** the planner-and-builder architecture, the house JSON structure and the endpoints (Chapter 3).
- **Backend development:** FastAPI endpoints, the Ollama client, the normaliser and the fallback generator.
- **Frontend development:** the 2D editor, 3D scene, walkthrough, planners, cost estimate and exports.
- **Interaction refinement:** click-to-place openings, drag handles, wall-length controls and the visual theme.
- **Testing:** service checks, end-to-end generation and the evaluation plan (Chapter 5).

4.2 IMPLEMENTATION

4.2.1 House Data Model

Every part of the system works on one JSON object, shown in Figure 4.1. Positions are in metres on the x-z ground plane, with y pointing up in the 3D scene. A wall is a segment from (x1, z1) to (x2, z2) with a height and thickness. A room is an axis-aligned rectangle. A door or window does not store coordinates of its own: it names its host wall through `wallId` and its place on that wall as a fraction from 0 to 1. The excerpt below is part of the actual response returned by `/generate` during the set-up test.

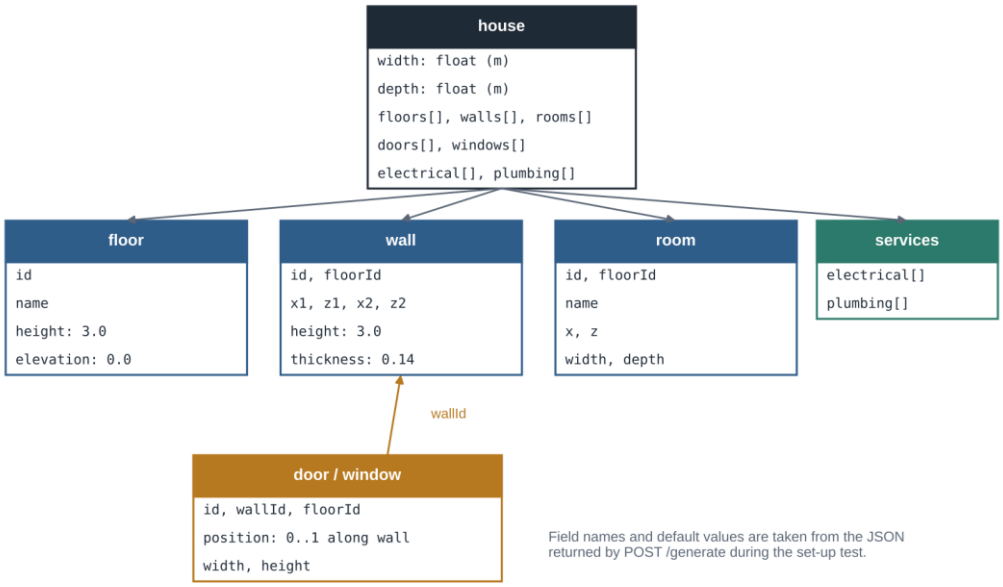


Figure 4.1: Structure of the house model

```
{ "width": 10.0, "depth": 8.0,
  "walls":  [ { "id": "wall1", "x1": 0.0, "z1": 0.0, "x2": 10.0, "z2": 0.0,
                "height": 3.0, "thickness": 0.14, "floorId": "f1" }, ... ],
  "doors":  [ { "id": "door1", "wallId": "wall1", "position": 0.5,
                "width": 0.9, "height": 2.1, "floorId": "f1" } ],
  "windows":[ { "id": "window1", "wallId": "wall3", "position": 0.5,
                "width": 1.2, "height": 1.2, "floorId": "f1" } ],
  "rooms":  [ { "id": "room1", "name": "Studio Room", "x": 0.0, "z": 0.0,
                "width": 10.0, "depth": 8.0, "floorId": "f1" } ],
  "floors": [ { "id": "f1", "name": "Ground Floor",
               "height": 3.0, "elevation": 0.0 } ],
  "electrical": [], "plumbing": [] }
```

Excerpt from the /generate response recorded during the set-up test (abridged)

Because openings refer to walls by identifier, moving or resizing a wall carries its doors and windows with it, and an opening whose wall no longer exists can be detected and removed.

4.2.2 Backend API Module

All endpoints are defined in `backend/main.py` [22]. Table 4.1 lists them.

Table 4.1: Backend API endpoints

Method and path	Request	Response
POST /generate	Natural-language prompt	House JSON with _source (model or fallback)
POST /modify	Instruction and current house JSON	Complete updated house JSON
POST /analyze	Current house JSON	List of advisory issues (room sizes)
GET /health	–	{"backend":"ok", "ollama":true, "model":"qwen2.5:7b-instruct", "model_installed":true}

CORS is open to all origins. This is acceptable for a single development machine but should be restricted before the backend is exposed on a network.

4.2.3 Local Language-Model Planner

The backend calls Ollama at 127.0.0.1:11434 with the model `qwen2.5:7b-instruct` and a context window (`num_ctx`) of 4,096 tokens [3], [4]. For generation, the prompt asks the model to describe the requested house as JSON in the structure of Figure 4.1. For modification, the current house is included in the prompt and the model may return up to 1,200 tokens. The reply is parsed tolerantly, so that a correct object surrounded by extra text is still accepted. Because the whole house is sent and returned on each edit, a large house can approach the context limit.

4.2.4 Normalisation and Fallback Module

`_single_floor_house()` cleans every model reply before it reaches the frontend: it coerces numbers and missing fields to usable values, places every element on a single ground floor, and drops doors and windows whose `wallId` does not match a wall. If the model is unreachable, times out or returns unusable text, `builtin_house()` builds a layout from keywords in the prompt without any model call. The response carries a `_source` field so that the two cases can be told apart; this matters for evaluation, because a fallback layout must not be counted as a model success.

4.2.5 2D Plan Editor Module

The 2D view is drawn as an architectural blueprint with a grid, tinted rooms, room-area badges and exterior dimension lines. The **Build and Add** tools work directly on the plan:

- **+ Wall:** click two points; a dashed preview with a live length readout follows the mouse. **Quick Wall** adds a wall and selects it immediately.
- **+ Door / + Window:** click any wall in the 2D or 3D view to place the opening at that point; a selected wall also offers "+ Door on this wall" and "+ Window on this wall".
- **Drag handles:** blue end handles stretch or rotate a wall, a green centre handle moves it, and a red handle slides a door or window along its wall.
- **Inspector:** openings have a position slider (5–95 %), ±5 % nudges, Centre and Flip, width, height and sill steppers, and reassignment to another wall. Walls have 0.5 m nudges, 90° rotation, height and thickness presets (0.10, 0.14, 0.18 and 0.23 m), and length controls in 0.5 m steps, ±0.5 m and ±1.0 m buttons, and "extend from end point" or "expand both ends" modes.

4.2.6 3D Scene and Walkthrough Module

The 3D view is built with Three.js through React Three Fiber and Drei [23], [24]. Each wall segment is extruded to its stored height and thickness, doors and windows are placed at their stored positions, and rooms, roof and materials are added from the same model. A floating control switches between the 3D view and the 2D blueprint, and a status badge shows the wall material and total floor area. In walkthrough mode the camera moves through the house at eye level so that the user can judge the spaces at human scale.

4.2.7 Services, Cost and Design-Check Module

Two planner modes add services to the model: the **Electrical Planner** for lights, switches and sockets, and the **Plumbing Planner** for sinks, toilets and drains. A cost estimate in Indian rupees is computed from the current model and updates as the design changes. Design checks are advisory: `/analyze` reports rooms whose sizes look unsuitable, and `validateHouse` in the frontend lists the problems it finds in the model; neither blocks or corrects it.

4.2.8 History, Persistence and Export Module

Every change pushes the previous house onto an undo history of up to 50 steps, so both manual and AI edits can be undone and redone. **Save** and **Load** keep projects in browser storage or as JSON files. The house can be exported as **JSON** (the full model), **CSV** (tabular design data) and **DXF** for CAD software such as AutoCAD; the current DXF writer outputs line entities only. **PDF / Print** uses the browser's print dialog.

4.3 MODULE INTEGRATION

Figure 4.2 shows how the modules cooperate for one AI request. The browser sends the prompt, and for a modification the current house, to the backend (1). The backend calls the model through Ollama (2, 3), parses and normalises the reply or falls back (4), and returns the house with its source (5). The frontend stores an undo snapshot, replaces its state (6) and redraws both views (7). The user can then ask for an analysis (8, 9).

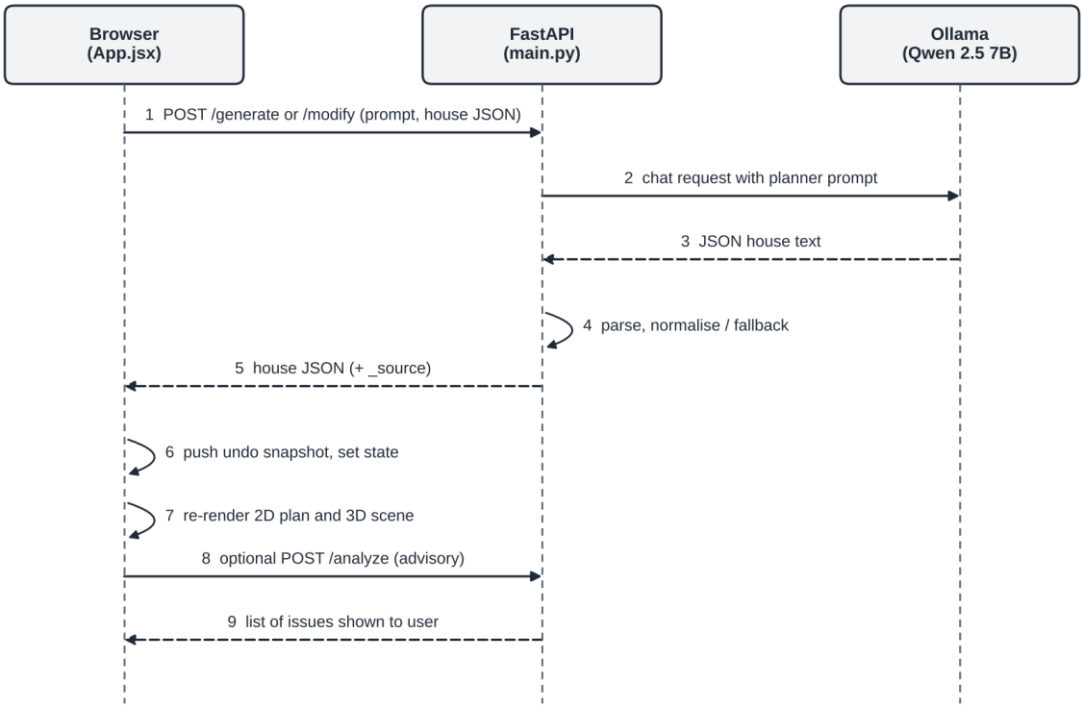


Figure 4.2: Frontend–backend interaction for an AI request

The two servers are started with `start_backend.bat` and `start_frontend.bat`. Errors are handled so that the user always has a model to work on: a failed generation falls back to the keyword generator, and the previous state remains available through undo. All traffic stays

on the local machine; the only external connections are the initial downloads of packages and the model.

CHAPTER 5 – RESULTS, CONCLUSION AND FUTURE WORK

5.1 RESULTS AND EVALUATION

The application was set up and run locally on the development laptop on 29 September 2026 and restarted on 22 and 28 October 2026. Table 5.1 records what was observed. These are observations of a working system, not formal test passes.

Table 5.1: Observed results from the running application

Check	Observed result	Figure
Ollama service	qwen2.5:7b-instruct installed and reachable at port 11434	–
GET /health	{"backend":"ok","ollama":true, "model":"qwen2.5:7b-instruct", "model_installed":true}	–
POST /generate (set-up test)	Well-formed single-floor house: 10 m × 8 m, 4 walls, 1 door, 1 window, 1 room	5.1
Frontend	Development server running at localhost:5173	5.2
Production build (npm run build)	Completed without errors	–
2D and 3D views of the starter 2BHK	Same house shown in both views	5.2, 5.3
Interactive editing	Click-to-place openings, drag handles and wall-length controls working in the browser	5.4
Walkthrough	First-person navigation inside the house	5.5

Figure 5.1 plots the JSON returned by `/generate` in the set-up test. The prompt used for that check was not logged, so the output is reported only as evidence that the pipeline returns a well-formed model whose openings lie on existing walls.

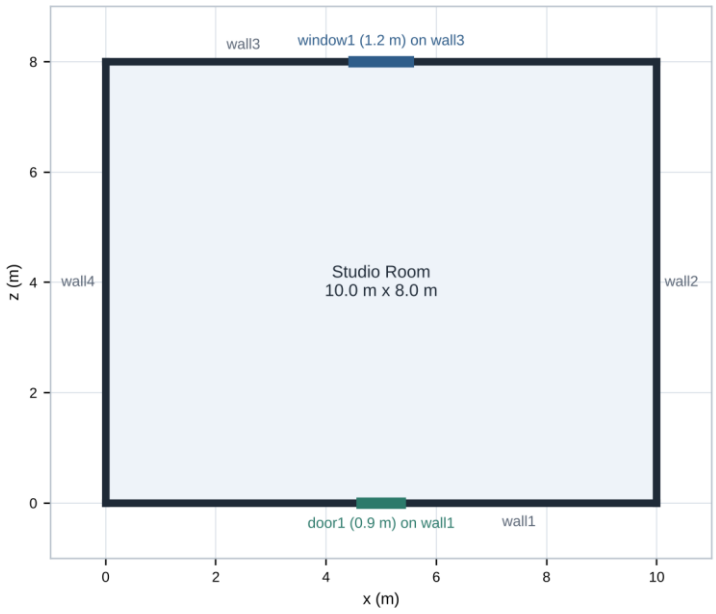


Figure 5.1: House model returned by /generate in the set-up test (plotted from the JSON response)

Figure 5.2: Main workspace with the starter 2BHK in the 2D blueprint view

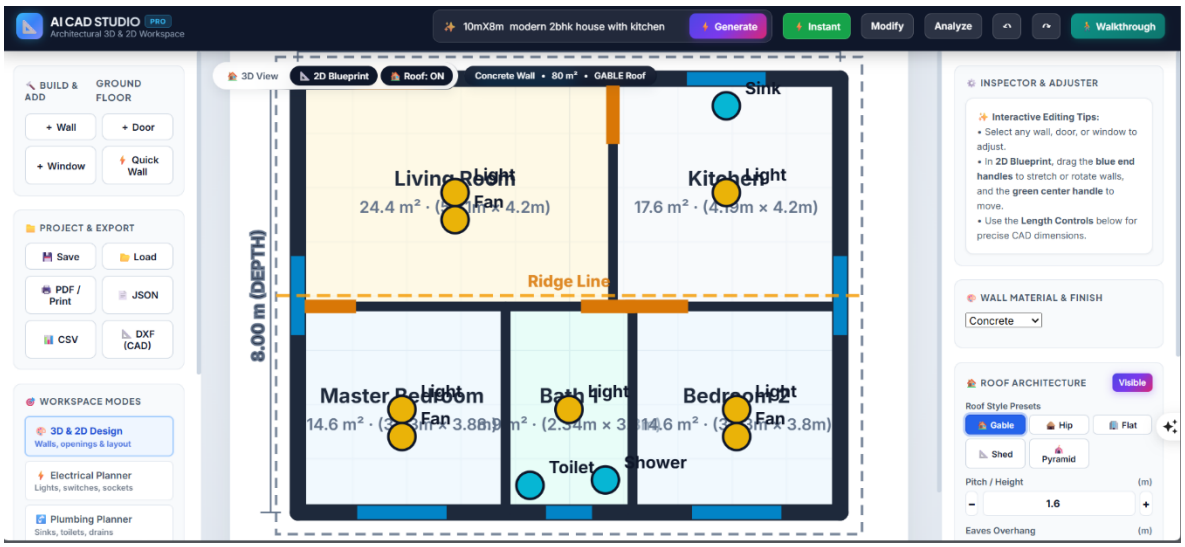


Figure 5.3: The same house in the 3D view

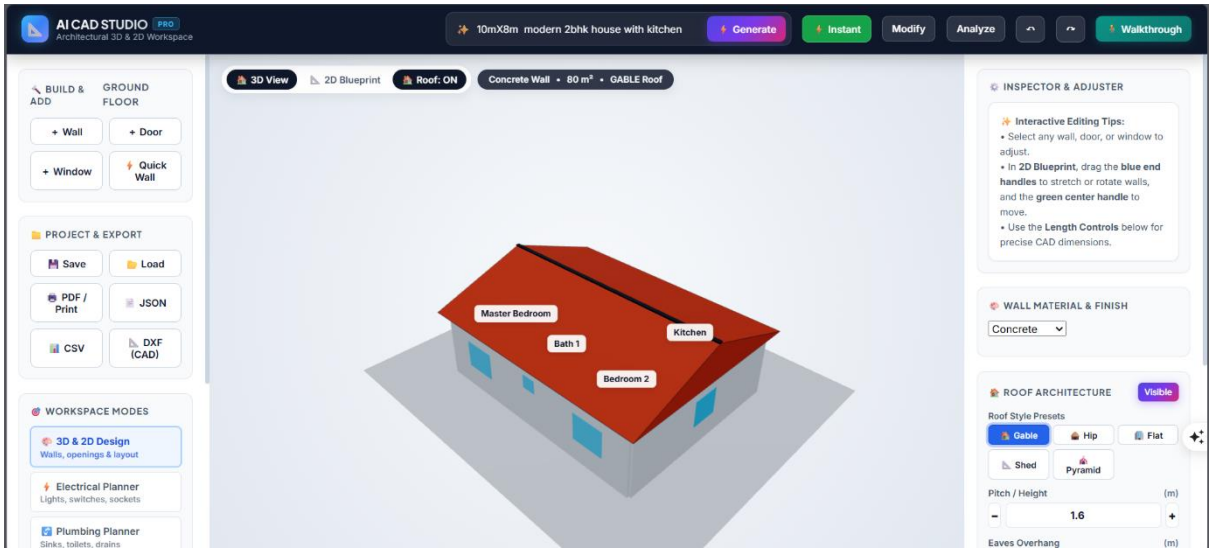


Figure 5.4: Inspector and drag handles on a selected wall



Evaluation Approach

The evaluation compares three configurations on the same prompts: **B1**, direct generation, in which the model returns the whole house; **B2**, the planner with typed tool calls but no validation; and **B3**, the planner with tool calls and blocking validation (the proposed method). Only B1 exists in the current code, so B2 and B3 require the Phase II tool layer. Each of the seven scenarios in Table 5.3 is run five times per configuration. Every response is logged with its `_source` field, and fallback layouts are counted as failures of the model. Table 5.2 defines the metrics.

Table 5.2: Evaluation metrics

Metric	Definition	Better
Tool Success Rate (TSR)	Operations executed without error ÷ operations planned × 100	Higher
Object Completion (OC)	Requested rooms, doors and windows present ÷ requested × 100	Higher
Dimensional Error (Ed)	Mean |requested – built| room dimension, in metres	Lower
Constraint Violations (V)	Overlaps, open walls and openings off walls per run	Lower
Edit Success (ES)	Edits that change the target without altering other rooms ÷ edits × 100	Higher
Time to Result (TTR)	Seconds from sending the prompt to a rendered model	Lower
Manual Effort (ME)	User corrections needed afterwards, counted by a fixed rule	Lower

Table 5.3 lists the planned test scenarios. Formal runs have not yet been recorded, so every status is pending; the team should run each case and replace the status with the measured outcome. The fourth column notes the behaviour expected from the current code.

Table 5.3: Test scenarios

ID	Scenario	Request	Expected from current code	Status
T1	Basic generation	Create a 2-bedroom house	House JSON from the model, normalised to one floor	Pending
T2	Larger house	Create a 3-bedroom house	As T1; longer output, closer to the context limit	Pending
T3	Unsupported request	Ask for a 2-floor house	Normaliser forces a single floor	Pending
T4	Add an opening	Add a window to Bedroom 2	Whole house returned; window kept only if its wallId exists	Pending
T5	Targeted edit	Make Bedroom 2 wider	Whole house returned; other rooms may also change	Pending
T6	Conflicting input	Give conflicting dimensions	No blocking check; /analyze may report issues	Pending
T7	Invalid reference	Refer to a non-existent room	No entity check; outcome depends on the model	Pending

Once the runs are complete, each metric will be reported as the mean over the five trials for each scenario and configuration. No accuracy, timing or effort figures are reported in this report, because none have been measured yet.

5.2 DISCUSSION

The observations show that the integrated system works end to end on one laptop: a prompt becomes a house model, the model is shown and edited in 2D and 3D, and the result can be saved and exported, with no cloud service involved. The set-up test also shows the value of the house JSON as a contract between the model and the code: openings refer to walls by identifier, so the normaliser can remove any that do not fit.

The same observations show where the design is not yet complete. The model still returns the whole house in one response, and every edit returns the whole house again, so a targeted change such as T5 is not guaranteed to leave other rooms untouched. Validation is advisory, so a conflicting request such as T6 is reported rather than prevented. The silent fallback keeps the user productive, but it would hide model failures from an evaluation unless `_source` is logged. These are exactly the points that the tool registry and blocking validator are designed to address [19], [20].

5.3 CHALLENGES AND LIMITATIONS

- **No tool layer yet:** the model returns complete houses rather than typed operations, so the B2 and B3 configurations cannot yet be run.
- **Advisory validation:** `/analyze` and `validateHouse` report problems but do not block or correct them.
- **Context size:** sending the whole house on every edit within a 4,096-token context limits the size of houses that can be modified reliably.
- **Single floor only:** the normaliser removes any second floor.
- **Model capability and hardware:** a 7B model runs locally but makes more planning errors than larger models, and CPU-only inference is slow.
- **Export depth:** the DXF writer outputs line entities only, without layers for doors, windows or labels.
- **Development settings:** CORS is open and there is no authentication, so the backend should run only on a trusted machine.
- **Testing:** formal test runs and timing measurements have not yet been recorded.

5.4 CONCLUSION

AI-CAD House Builder demonstrates a locally run approach to natural-language house modelling. A React and Vite workspace, a Three.js scene, a FastAPI backend and a Qwen 2.5 7B model served by Ollama have been integrated into a working application that generates a single-floor house from a sentence, lets the user edit it by hand or by instruction, shows it in 2D, 3D and walkthrough, estimates its cost and exports it. Keeping the model in the role of planner and building geometry in code gives a model that stays editable and consistent across every view, and running locally keeps designs private. The next step is to put a typed tool

registry and a blocking validator between the planner and the model, and then to measure the benefit with the evaluation defined in this chapter.

5.5 FUTURE WORK

- **Tool layer:** typed operations such as add_wall, add_door, add_window, resize_room, set_roof and add_fixture, so that the model plans changes rather than rewriting the house.
- **Blocking validator:** check each operation for overlaps, open walls and openings off walls, and return errors the model can retry on.
- **Measured evaluation:** add switches for B1, B2 and B3, log `_source`, and run T1–T7 five times each.
- **Runtime:** move to llama.cpp with a quantised Qwen 2.5 7B model as specified in the design [21].
- **Richer output:** multi-floor houses with stairs, layered DXF and BIM/IFC export, and furniture libraries.
- **Hardening:** restrict CORS and add accounts if the tool is ever served beyond one machine.

REFERENCES

[1] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," ACM Computing Surveys, vol. 55, no. 12, Art. no. 248, 2023.

[2] K. Alrashedy, P. Tambwekar, Z. Zaidi, M. Langwasser, W. Xu, and M. Gombolay, "Generating CAD code with vision-language models for 3D designs," in Proc. Int. Conf. Learning Representations (ICLR), 2025.

[3] Qwen Team (A. Yang et al.), "Qwen2.5 technical report," arXiv preprint arXiv:2412.15115, 2024.

[4] Ollama, "Ollama: Get up and running with large language models locally." [Online]. Available: https://github.com/ollama/ollama

[5] N. Nauata, S. Hosseini, K.-H. Chang, H. Chu, C.-Y. Cheng, and Y. Furukawa, "House-GAN++: Generative adversarial layout refinement network towards intelligent computational agent for professional architects," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2021, pp. 13632–13641.

[6] M. A. Shabani, S. Hosseini, and Y. Furukawa, "HouseDiffusion: Vector floorplan generation via a diffusion model with discrete and continuous denoising," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2023, pp. 5466–5475.

[7] Meta Open Source, "React documentation." [Online]. Available: https://react.dev

[8] Vite, "Vite documentation." [Online]. Available: https://vite.dev

[9] W. Wu, X.-M. Fu, R. Tang, Y. Wang, Y.-H. Qi, and L. Liu, "Data-driven interior plan generation for residential buildings," ACM Transactions on Graphics, vol. 38, no. 6, 2019.

[10] N. Nauata, K.-H. Chang, C.-Y. Cheng, G. Mori, and Y. Furukawa, "House-GAN: Relational generative adversarial networks for graph-constrained house layout generation," in Proc. European Conf. Computer Vision (ECCV), 2020.

[11] R. Hu, Z. Huang, Y. Tang, O. van Kaick, H. Zhang, and H. Huang, "Graph2Plan: Learning floorplan generation from layout graphs," ACM Transactions on Graphics, vol. 39, no. 4, Art. no. 118, 2020.

[12] J. Sun, W. Wu, L. Liu, W. Min, G. Zhang, and L. Zheng, "WallPlan: Synthesizing floorplans by learning to generate wall graphs," ACM Transactions on Graphics, vol. 41, no. 4, 2022.

[13] S. Leng, Y. Zhou, M. H. Dupty, W. S. Lee, S. Joyce, and W. Lu, "Tell2Design: A dataset for language-guided floor plan generation," in Proc. 61st Annual Meeting of the Association for Computational Linguistics (ACL), 2023, pp. 14680–14697.

[14] R. Wu, C. Xiao, and C. Zheng, "DeepCAD: A deep generative network for computer-aided design models," in Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV), 2021, pp. 6772–6782.

[15] M. S. Khan, S. Sinha, T. U. Sheikh, D. Stricker, S. A. Ali, and M. Z. Afzal, "Text2CAD: Generating sequential CAD designs from beginner-to-expert level text prompts," in Advances in Neural Information Processing Systems, vol. 37, 2024, pp. 7552–7579.

[16] J. Li, W. Ma, X. Li, Y. Lou, G. Zhou, and X. Zhou, "CAD-Llama: Leveraging large language models for computer-aided design parametric 3D model generation," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2025, pp. 18563–18573.

[17] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in Advances in Neural Information Processing Systems, vol. 36, 2023.

[18] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in Proc. Int. Conf. Learning Representations (ICLR), 2023.

[19] L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Callan, and G. Neubig, "PAL: Program-aided language models," in Proc. 40th Int. Conf. Machine Learning (ICML), PMLR, vol. 202, pp. 10764–10799, 2023.

[20] D. Mallis, A. S. Karadeniz, S. Cavada, D. Rukhovich, N. Foteinopoulou, K. Cherenkova, A. Kacem, and D. Aouada, "CAD-Assistant: Tool-augmented VLLMs as generic CAD task solvers," in Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV), 2025, pp. 7284–7294.

[21] G. Gerganov et al., "llama.cpp: LLM inference in C/C++." [Online]. Available: https://github.com/ggml-org/llama.cpp

[22] S. Ramírez, "FastAPI documentation." [Online]. Available: https://fastapi.tiangolo.com

[23] three.js authors, "three.js documentation." [Online]. Available: https://threejs.org/docs

[24] Poimandres, "React Three Fiber documentation." [Online]. Available: https://r3f.docs.pmnd.rs